

How We Tuned the PID — Method, Reasoning, and Every Problem We Hit

HACKSIMUL8 2026 · 6-DOF UAV Quadcopter

"What tuning method are you using, and how are you tuning the model?"

This document answers that in full, including the problems encountered and how each was resolved. It is written to be read start to finish by someone who was not in the room.

The short answer

Four tuning methods were implemented and compared. The one we use is ITAE optimisation on the full nonlinear model.

Final gains: **Kp = 30.1847, Ki = 0.0000, Kd = 10.3994** Settling 0.98 s · overshoot 0.00 % · lowest ITAE of every design tested, including all three from the reference paper.

But the gains are not the interesting part. **The problem statement explicitly grades the methodology:**

"Neat description of the methodology used for tuning of PID controller gains should be presented during the evaluation. Use the best method available for tuning."

So the rest of this document is the *why*.

Step 1 — Understand the plant before touching a controller

The altitude dynamics are:

$$\ddot{z} = -g + (U1/m) \cdot \cos \phi \cdot \cos \theta$$

Two properties matter:

(a) It is nonlinear. Thrust is multiplied by $\cos \phi \cdot \cos \theta$. Tilt the quadcopter and it loses lift — 4.47 % of it at 17° of pitch.

(b) It is a double integrator. Linearised, $G(s) = 1/(m s^2) = 1/(0.516 s^2)$. **Both poles sit at the origin.**

That second point has three consequences we had to design around:

1. The system is **marginally stable** — a constant force makes altitude grow as t^2 .
2. **Proportional control alone cannot stabilise it.** With P-only, the closed loop is $m \cdot s^2 + Kp = 0$, giving poles at $\pm j\sqrt{Kp/m}$ — purely imaginary, so it oscillates forever at *any* gain. **Derivative action is mandatory, not optional.**
3. **The Ziegler–Nichols ultimate-gain method is degenerate** — see Step 3.

Step 2 — The design decision that shapes everything: feedback linearisation

The obvious approach is to linearise about hover and treat tilt as a disturbance the controller has to reject. **We do something stronger: cancel the nonlinearity exactly.**

Command the thrust as:

$$U1 = m \cdot (g + u_z) / (\cos \phi \cdot \cos \theta)$$

Substitute back into the plant:

$$\begin{aligned}\ddot{z} &= -g + (1/m) \cdot [m(g + u_z)/(\cos \phi \cos \theta)] \cdot \cos \phi \cos \theta \\ &= -g + g + u_z \\ &= u_z\end{aligned}\qquad \leftarrow \text{exactly, no approximation}$$

The trigonometry cancels **completely**, at **any** tilt angle. The PID now controls a clean $1/s^2$.

Term	What it does
$m \cdot g$	gravity feed-forward — the PID starts from zero instead of fighting weight
$1/(\cos \phi \cos \theta)$	tilt compensation — restores exactly the lift lost to tilting

Measured result:

Pitch angle	Altitude sag
8.6°	0.000034 m
17.2°	0.000134 m
25.8°	0.000298 m

The reference paper's altitude law (their Eq. 27) is a plain PID with neither term. Under the same 17.2° pitch step, their best-rated design sags **0.034 m — 253× more than ours**.

Step 3 — Why the textbook method does not apply here (and the qualification that matters)

Most teams will reach for **Ziegler–Nichols**. On this plant, as specified, it does not work:

- **Ultimate-gain (closed-loop) method** needs a gain K_u at which the loop sustains a constant-amplitude oscillation. For a pure double integrator it oscillates at every $K_p > 0$, so **no unique K_u exists**.
- **Reaction-curve (open-loop) method** needs an S-shaped step response to read a delay and time constant from. A double integrator's step response is a diverging parabola — **no S-curve**.

⚠ The qualification — do not overstate this

The reference paper applies Ziegler–Nichols successfully to this same quadcopter, and publishes measured values ($k_{th} = 124.99$, $\tau_{th} = 3.52$ s).

We initially wrote that ZN was simply "undefined for this plant". **That was wrong**, and we corrected it after reading the paper.

The difference is in the *model*, not the method. Their Eq. (6) retains air resistance:

$$m \cdot \ddot{z} + m \cdot g + A_z \cdot \dot{z} = F \cdot \cos \phi \cos \theta$$

With $A_z \neq 0$, one pole moves from the origin to $-A_z/m$, the plant becomes $1/(s(ms+A_z))$, and a finite ultimate gain exists.

But Table 1 — in the problem statement and in the paper — gives no values for A_x , A_y , A_z . Setting them to zero is the only consistent reading of the data supplied, and that is exactly what makes our altitude channel a pure double integrator.

The correct sentence: "For the plant as specified by Table 1, the ZN ultimate-gain method is degenerate, because with no drag coefficients given the altitude channel is a pure double integrator. The reference avoids this because its model retains aerodynamic drag."

That engages with the source instead of contradicting it. A judge who has read the paper would have caught the unqualified version immediately.

Step 4 — The four methods, as a progression

Each step relaxes an assumption the previous one made.

Method A — Analytical pole placement

A PID acting on $\ddot{z} = u_z$ gives a **third-order** closed loop:

$$s^3 + K_d \cdot s^2 + K_p \cdot s + K_i = 0$$

Match it to a dominant complex pair plus a faster real pole at $-p$:

$$(s^2 + 2\zeta\omega_n s + \omega_n^2)(s + p)$$

$$\rightarrow K_d = 2\zeta\omega_n + p \quad K_p = \omega_n^2 + 2\zeta\omega_n p \quad K_i = \omega_n^2 \cdot p$$

Specification: $\zeta = 0.9$, settling 2 s $\rightarrow \omega_n = 2.222$ rad/s , $p = 4.0$.

Result: $K_p = 20.94$, $K_i = 19.75$, $K_d = 8.00$ — ITAE **0.5311**

Problem hit: the first version designed K_p and K_d as a PD, then bolted K_i on afterwards. That produced **20 % overshoot with no settling**. Integral action *moves the closed-loop poles*, destroying the damping just designed for. **Fix:** place all three poles simultaneously.

Remaining limitation, worth explaining: Method A still overshoots 26 %. Pole placement controls **poles**, but a PID also introduces a **zero** at $s = -K_i/K_p \approx -0.94$. A slow zero causes overshoot regardless of where

the poles are. Pole placement is blind to it.

Method B — `pidtune`

MATLAB's automatic tuner on the linear plant.

Result: $K_p = 0.19$, $K_i = 0.01$, $K_d = 0.97$ — ITAE **11.1617**, PM 69°

Very conservative: crossover of only 1 rad/s, giving a rock-solid phase margin but a 6.5 s rise time. `pidtune` defaults to caution on a marginally stable plant.

Method C — `pidtune` with a bandwidth target

Same tuner, crossover specified by us at 2.5 rad/s.

Result: $K_p = 1.16$, $K_i = 0.14$, $K_d = 2.43$ — ITAE **5.0895**

Demonstrates trading speed against stability *deliberately* rather than accepting a default.

Method D — ITAE optimisation on the nonlinear model ← **what we use**

Minimise a performance index on the **full nonlinear simulation**, with actuator saturation and anti-windup active:

```
J = ITAE
    + 0.30 * (overshoot %)
    + 0.02 * (peak thrust above hover)
    + 0.002 * (total control variation)
    + 2e-4 * (Kp2 + Kd2)
```

Minimised with `fminsearch` — no Optimization Toolbox required.

Result: $K_p = 30.1847$, $K_i = 0.0000$, $K_d = 10.3994$ — ITAE **0.0840**

Why this is "the best method available": A, B and C all optimise a *linear approximation*. Method D optimises **what actually flies** — the nonlinearity, the actuator limits, and the discrete 100 Hz controller.

Step 5 — Why each cost term is there

Every weight was added in response to a specific failure.

Term	Weight	Why it exists
ITAE	1.0	Time-weighted absolute error. Punishes errors that <i>persist</i> while tolerating the unavoidable transient after a step — the right index for tracking.
Overshoot	0.30	For an altitude controller, overshoot is a collision risk , not cosmetic.
Peak thrust	0.02	Discourages designs that live permanently in saturation.
Control variation	0.002	Discourages actuator chatter.
Gain regularisation	2e-4	Added after a real failure — see below.

The problem that forced gain regularisation

With an unknown payload, `fminsearch` discovered it could mask a steady-state error with **Kp ≈ 1978** rather than using integral action. Mathematically it minimised the cost; physically it is useless — such gains amplify sensor noise and sit permanently in saturation.

Adding $2e-4 \cdot (Kp^2 + Kd^2)$ expresses the real engineering constraint, and pushed the optimiser toward using **Ki** for steady-state error, which is what Ki is for.

Step 6 — The results

All four evaluated on the identical nonlinear model with saturation:

Method	Kp	Ki	Kd	Tr [s]	Ts [s]	OS [%]	ITAE
A: Pole placement	20.938	19.753	8.000	0.420	2.830	26.21	0.5311
B: pidtune	0.186	0.009	0.974	6.480	—	0.00	11.1617
C: pidtune @ wc = 2.5	1.160	0.136	2.435	2.590	—	15.88	5.0895
D: ITAE optimised	30.185	0.000	10.399	0.560	0.950	0.00	0.0840

— means the response never entered the $\pm 2\%$ band inside the simulation window.

Benchmarked against the reference paper

Our implementation of their tuning formulas reproduces their **published** gains to under 4×10^{-5} — done *before* any comparison, to confirm we had read the paper correctly:

Method	Their published (Kp, Ki, Kd)	Our computed
Ziegler–Nichols	74.9940, 42.6102, 32.9974	identical
Tyres–Luyben	56.8136, 7.3365, 31.7435	identical

Then, all four designs on the identical plant at their own setpoint of 2.0 m:

Design	Ts [s]	OS [%]	ITAE	Tilt sag [m]
Ziegler–Nichols (theirs)	4.610	12.13	1.7259	0.004522
Tyreus–Luyben (theirs)	10.300	4.96	6.6841	0.034002
MATLAB PID Tuner (theirs)	1.490	0.00	0.3319	0.008033
Ours	0.980	0.00	0.1874	0.000134

Lowest ITAE of all four — 1.8× better than the best of theirs, 36× better than Tyreus–Luyben, the method their paper concludes is best.

Separating the two improvements — apples to apples

The table above mixes **two** distinct improvements: a better tuning method *and* a better controller structure. To separate them, we re-tuned with our ITAE method but **without** feedback linearisation — i.e. exactly the paper's Eq. (27) structure. Any remaining difference is attributable to the **tuning alone**:

Design (all plain PID)	Kp	Ki	Kd	Ts [s]	OS [%]	ITAE
Ziegler–Nichols (Mien & Tu)	74.994	42.610	32.997	4.610	12.13	1.7259
Tyreus–Luyben (Mien & Tu)	56.814	7.336	31.743	10.300	4.96	6.6841
MATLAB PID Tuner (Mien & Tu)	57.005	0.001	23.102	1.490	0.00	0.3319
OURS: ITAE opt, plain PID	21.395	0.000	8.544	1.080	0.03	0.2307

On an identical controller structure:

- **1.44× lower ITAE** than their best
- **28% faster settling** — 1.080 s vs 1.490 s
- **2.7× smaller gains** (Kp 21.4 vs 57.0), meaning less noise amplification and less saturation

So the improvement decomposes as:

Source	ITAE
Their best design	0.3319
→ our tuning method , same structure	0.2307
→ plus feedback linearisation	0.1874

A fairness caveat, stated plainly. We optimise ITAE and then report ITAE, so that metric is partly circular. **Settling time is the independent check** — it appears nowhere in our cost function, and we are still 28% faster.

Where we are not better: they control all six degrees of freedom in a cascade; we do altitude only, which is what Task 2 asked for. Their PID Tuner design also edges us on overshoot (0.00% vs 0.03%). And their heuristic formulas are far simpler to apply in the field — ours needs a numerical optimiser and a validated simulation model.

Cross-check on our reproduction: their paper reports Tyreus–Luyben achieving "steady-state error of less than 1 %". Our simulation of their gains gives **0.592 %** — inside their stated bound.

Step 7 — Two results that look wrong but are not

(a) Method D returned $K_i = \text{exactly } 0$

Feedback linearisation already cancels gravity via the $m \cdot g$ feed-forward, and the plant is exactly $1/s^2$. In the **nominal** case there is no steady-state error for integral action to remove — it would only add overshoot. The optimiser correctly deleted it.

The consequence matters more than the result. With $K_i = 0$ the controller is **not robust** to an unknown payload:

Payload	Steady-state error
×1.0	0.000 m
×1.4	0.120 m
×1.8	0.239 m
×2.2	0.359 m

This produced a prediction, written down before any Task 3 data existed:

The ML model should learn to reintroduce integral action when it detects a payload.

Task 3 later confirmed it exactly — optimal K_i came out $0 \rightarrow 5.50 \rightarrow 11.01 \rightarrow 16.05$ as mass went $\times 1.0 \rightarrow \times 2.2$.

(b) Thrust saturates at exactly 20.248 N

That is the rotors' real limit, and hitting it is correct behaviour. Method D demands about 30 m/s^2 while the rotors deliver at most 29.43, so the command clips briefly during the initial climb. **Clamping anti-windup** prevents the integrator winding up during that interval, which is why overshoot is still 0.00 %.

Step 8 — Engineering realism in the simulation

A tuning result is only as credible as the model it was tuned against. All of these are active in `sim_altitude.m` and in the Simulink model:

Feature	Why it matters
Actuator saturation	thrust clipped to $[0, 20.248]$ N
Clamping anti-windup	integrator frozen while saturated
Derivative on measurement	no derivative kick on a step command
Discrete controller at 100 Hz	matches real flight hardware
RK4 plant integration	accurate between control updates

Every problem we hit, and how it was resolved

#	Problem	Symptom	Resolution
1	PD designed first, Ki added after	20 % overshoot, never settled	place all three closed-loop poles simultaneously
2	ZN claimed "undefined" without qualification	contradicted by the cited paper	qualified: degenerate <i>for the plant as specified by Table 1</i> ; the paper retains drag terms
3	Optimiser brute-forced steady-state error	$K_p \approx 1978$	added gain regularisation $2e-4(K_p^2 + K_d^2)$
4	<code>fminsearch</code> never explored Ki	$K_i \approx 0$ everywhere, error persisted	seed Ki at 5.0 — its simplex barely perturbs a component starting at exactly zero
5	Overshoot weight too low	learned controller flew to 1.62 m on a 1.00 m command	raised 0.05 → 0.30 and re-labelled the dataset
6	Cost change shifted the Task 2 gains 8 %	slides would disagree with a live run	regenerated Task 3 from scratch; rebuilt the Simulink model

Problem 4 deserves emphasis. `fminsearch` builds its initial simplex by perturbing each component of the starting point. A component starting at *exactly zero* gets an almost invisible first step, so integral action was never meaningfully explored. A mundane numerical detail was hiding the single best result in the project.

How to reproduce all of this

```
cd 'C:\Users\LENOVO\Downloads\HACKSIMU8\solution'
task2_altitude_pid          % all four methods, comparison table  (~60 s)
task2b_zn_tyreus_luyben     % benchmark vs the reference paper   (~30 s)
verify_all                  % 25 independent checks              (~2 min)
```

Relevant file	Role
<code>task2_altitude_pid.m</code>	The four tuning methods
<code>task2b_zn_tyreus_luyben.m</code>	The reference-paper benchmark
<code>alt_cost.m</code>	The objective Method D minimises — every weight documented inline
<code>sim_altitude.m</code>	The nonlinear simulator: saturation, anti-windup, discrete control
<code>perf_metrics.m</code>	Tr, Ts, OS, SSE, ITAE, IAE, ISE, control effort

If asked in one sentence

"We compared four methods — analytical pole placement, `pidtune`, `pidtune` at a specified bandwidth, and ITAE optimisation on the full nonlinear model with saturation and anti-windup. The last is what we use, because the other three optimise a linear approximation while it optimises what actually flies. It gives the lowest ITAE of every design we tested, including all three published in the reference paper."